Boring but important disclaimers:

- If you are not getting this from the GitHub repository or the associated Canvas page (e.g. CourseHero, Chegg etc.), you are probably getting the substandard version of these slides Don't pay money for those, because you can get the most updated version for free at

  `https://github.com/julianmak/OCES4303_ML_ocean`

  The repository principally contains the compiled products rather than the source for size reasons.
- Associated Python code (as Jupyter notebooks mostly) will be held on the same repository. The source data however might be big, so I am going to be naughty and possibly just refer you to where you might get the data if that is the case (e.g. JRA-55 data). I know I should make properly reproducible binders etc., but I didn't...
- I do not claim the compiled products and/or code are completely mistake free (e.g. I know I don't write Pythonic code). Use the material however you like, but use it at your own risk.
- As said on the repository, I have tried to honestly use content that is self made, open source or explicitly open for fair use, and citations should be there. If however you are the copyright holder and you want the material taken down, please flag up the issue accordingly and I will happily try and swap out the relevant material.

OCES 4303 :

an introduction to data-driven and ML methods in ocean sciences

Session 8: Convolutional Neural Networks

Outline

- basics of **PyTorch**

  → redoing last time's MLPs

- convolutions

  → kernels and pooling

  → resizing issues

  → examples

- Convolutional Neural Networks (CNNs)

  → TL;DR: kernel **weights** as part of control variables

  → classification and regression demonstration

Figure: Convolved ad-hoc TA.

Oceanic application

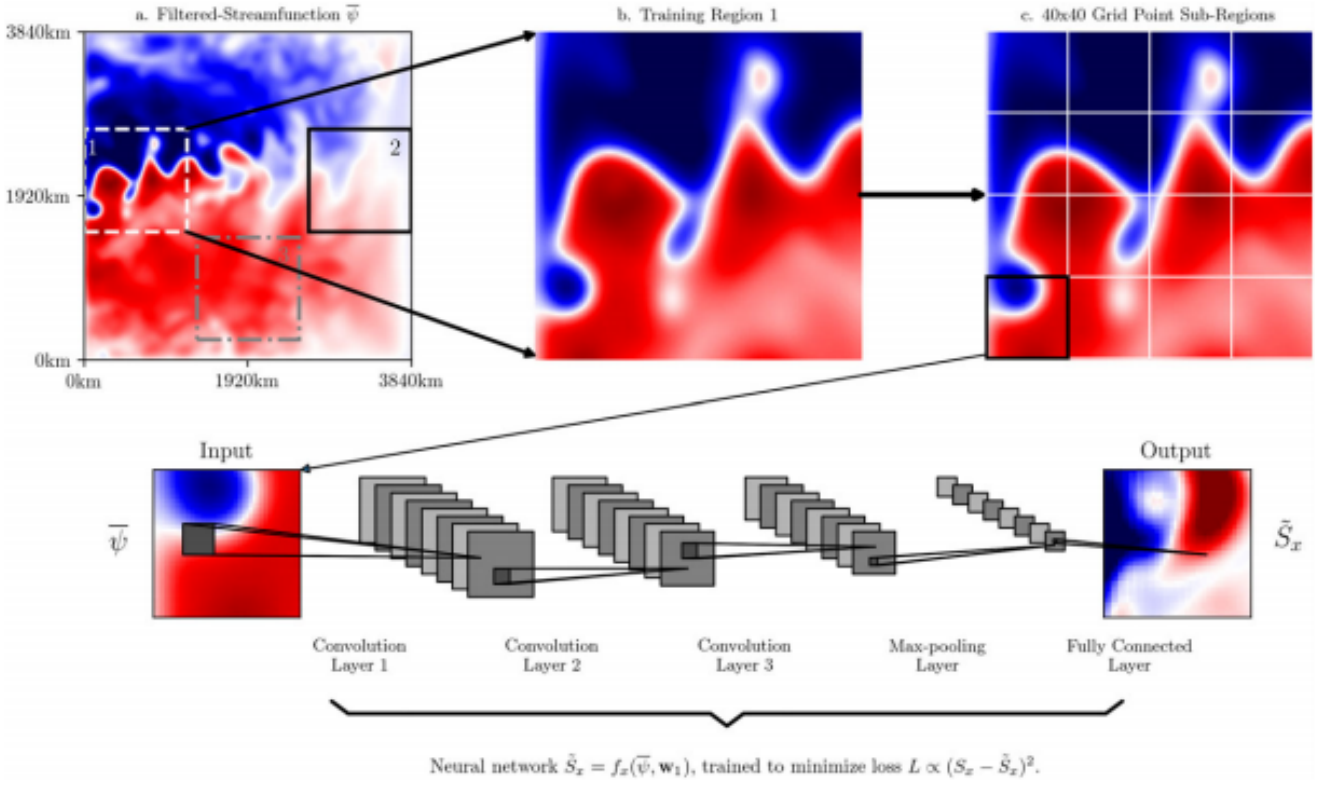


Figure: CNN applied to an eddy parameterisation problem. From Bolton & Zanna (2019).

- eddy parameterisation problem

  $\rightarrow$ regression problem: predict one image from another

Oceanic application

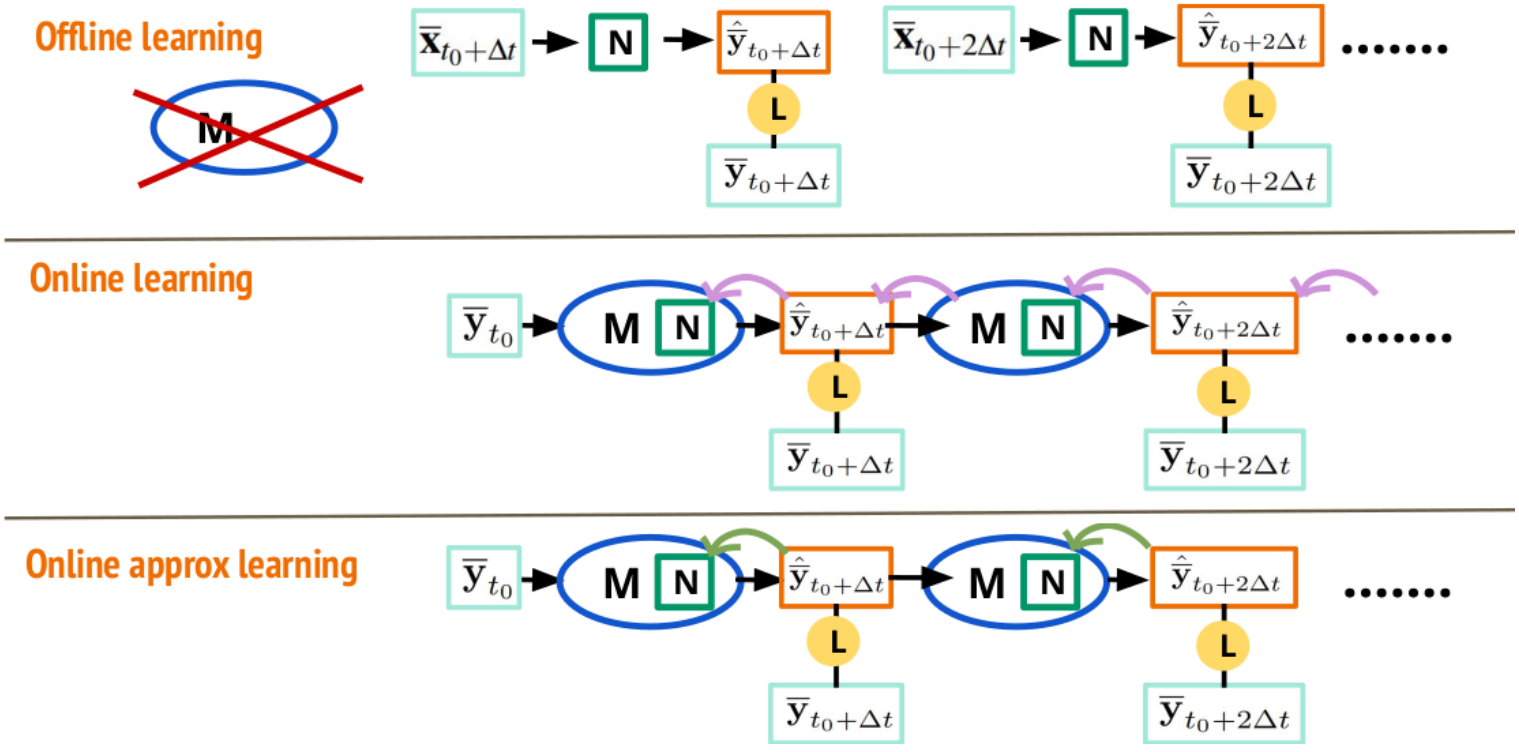


Figure: CNN training nested inside an eddy parameterisation problem. From Yan *et al.* (2025).

- as before, but nesting a CNN within a model (cf. RNN next time)

  → related to variational data assimilation approaches

PyTorch

- PyTorch and TensorFlow the two most popular libraries for neural networks

  → underlying them are procedure to manipulate tensors

  (multi-dim arrays)

  → links up to solvers and loss functions accordingly

  → have interfaces if need be (e.g. Lightning and Keras)
- JAX (e.g. w/ Keras) could be used in principle

PyTorch: building MLP as last time

- need to convert arrays into **tensor** objects

  → so PyTorch library knows what to do with them

  → extra useful functionalities (e.g. reshaping, resizing after manipulation)

- `FloatTensor`: floats (but `float32`)
- `LongTensor`: signed integers
- `ByteTensor`: unsigned integers

- in principle will do 'sane' things, but best be explicit

  → e.g. `torch.tensor([1., 1., 1.])` will default to `FloatTensor`

  → e.g. `torch.tensor([1, 1, 1])` will default to `ByteTensor`, will **fail** when passed to `conv1d`

PyTorch: building MLP as last time

- specify neural network structure

```
# define the model architecture

class nn_classify_pets(nn.Module):
    def __init__(self):  # specify input dims below
        super(nn_classify_pets, self).__init__()

        # nest it in one go
        self.layers = nn.Sequential(
            nn.Linear(64**2, 100),  # image is 64**2, 100 nodes (sklearn default)
            # nn.ReLU(),            # no activation function
            nn.Linear(100, 2)       # two possible outputs
        )

    # actual model structure: input -> hidden layer -> outputs, ReLU on input and hidden
    def forward(self, x):
        out =  self.layers(x)
        return out
```

- define as a class, need `__init__`
- specify network structure
- need a `forward` to specify the feed-forward

  $\rightarrow$ this is for internal taping/annotation for doing back-propagation later

PyTorch: building MLP as last time

- define loss function

  → for binary classification we can choose `CrossEntropy`

  → others would work too

- define optimiser

  → use Adam again

  → need to initialise model (e.g. `model = MLP()`) and pass model parameters to the optimiser

  → telling the optimiser what are the control variables

- sequence of things to do:

  → do a feed-forward

  → evaluate loss function

  → back-propagate

  → update parameters, repeat

PyTorch: building MLP as last time

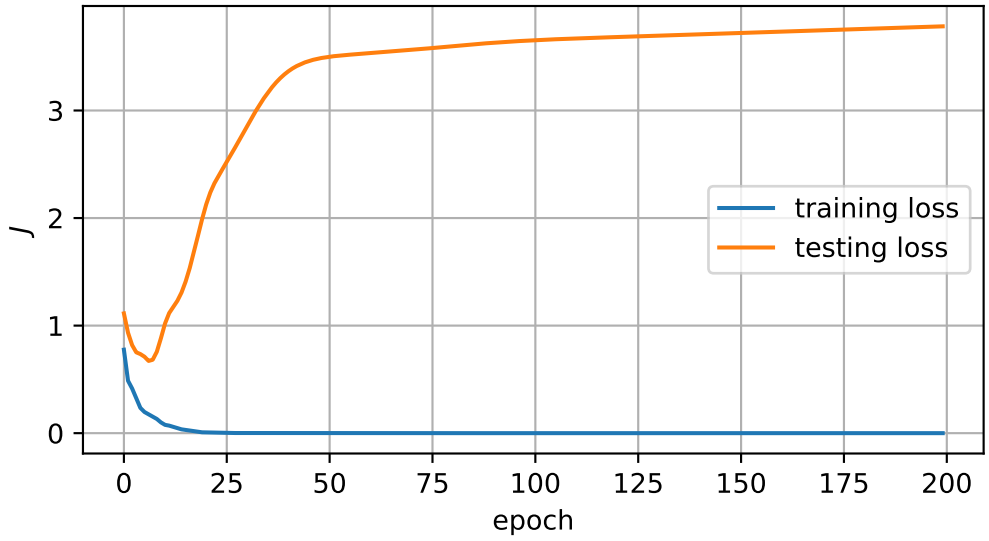
```
for epoch in range(num_epochs):

    # iteration step
    optimizer.zero_grad()  # clear gradients if it exists (from loss.backward())
    Y_pred = model(X_train)  # feed-forward
    J_train = J(Y_pred, Y_train) # compute loss
    J_train.backward()  # back propagation
    optimizer.step()  # iterate

    # diagnostics: evaluation of metrics as we go along
    Y_pred = model(X_test)
    J_test = J(Y_pred, Y_test)
    train_J[epoch] = J_train.item()
    test_J[epoch] = J_test.item()

    if (epoch + 1) % out_epoch == 0:
        print(f"Epoch {epoch+1}/{num_epochs}, "
            + f"Train Loss: {J_train.item():.4f}, "
            + f"Test Loss: {J_test.item():.4f}")
```

- above subroutine chains it all up, iterates over `epochs`

  $\rightarrow$ safety call to clear some calculations to avoid contamination

  $\rightarrow$ extra diagnostic calls

PyTorch: building MLP as last time



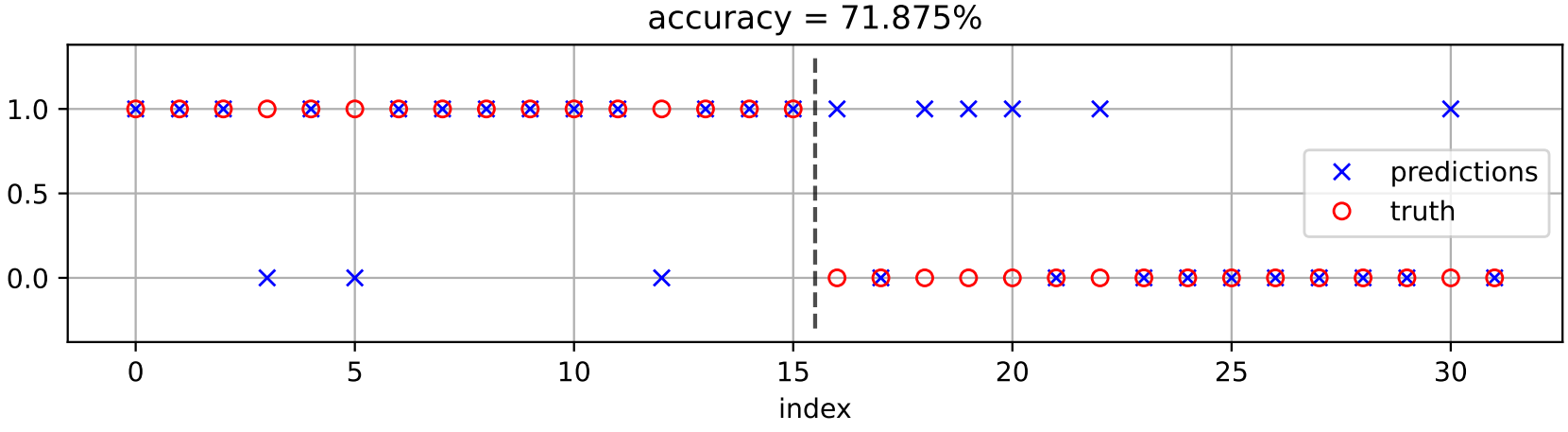


Figure: Perceptron with two hidden layers, no activation (cf. one of the cases considered last time but with `sklearn.MLPClassifier`).

CNNs

- Convolutional Neural Networks (CNNs)

  $\rightarrow$ has slightly better control on number of degrees of freedom compared to fully connected MLPs

  $\rightarrow$ naturally suited to images

CNNs

- Convolutional Neural Networks (CNNs)

  $\rightarrow$ has slightly better control on number of degrees of freedom compared to fully connected MLPs

  $\rightarrow$ naturally suited to images

- should be self-explanatory why after going through what convolutions are...

  $\rightarrow$ recall from OCES 3301 where convolutions were used in relation to **filtering** in time series data:

$$(f * G)(x) = \int f(x')G(x, x')\, \mathrm{d}x'$$

  where instead of (x, x') we had (t, τ)

  $\rightarrow$ easier in the discrete case with an example...

Convolutions

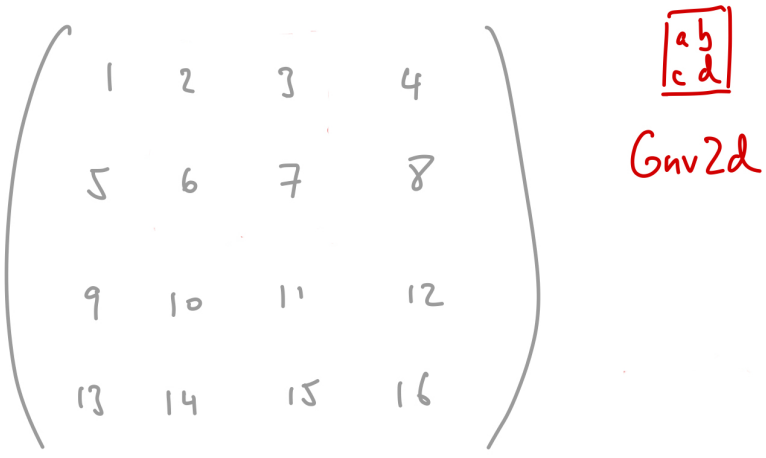


Figure: Sample array to be convolved with a kernel (the red stuff). Kernel here is of size (2,2).

Convolutions

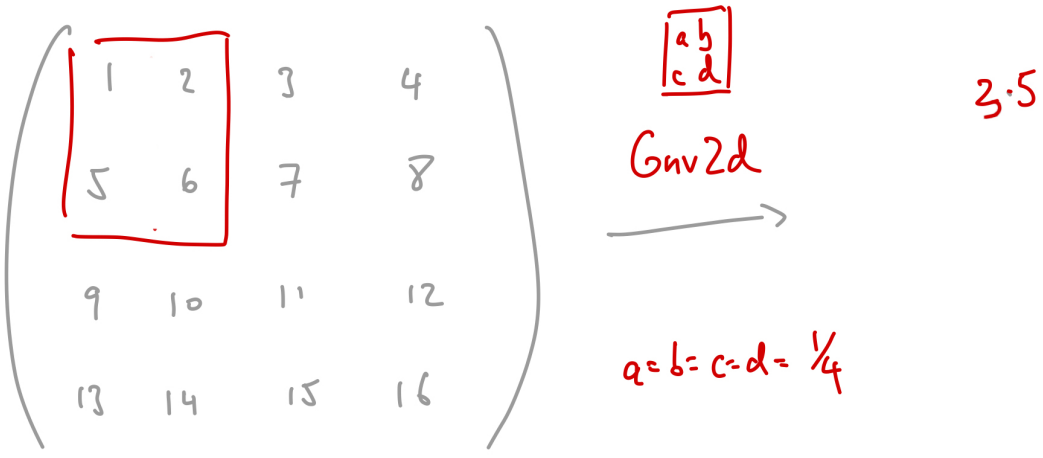


Figure: Choose a kernel, throw down kernel, element-wise multiplication, then sum.

Convolutions

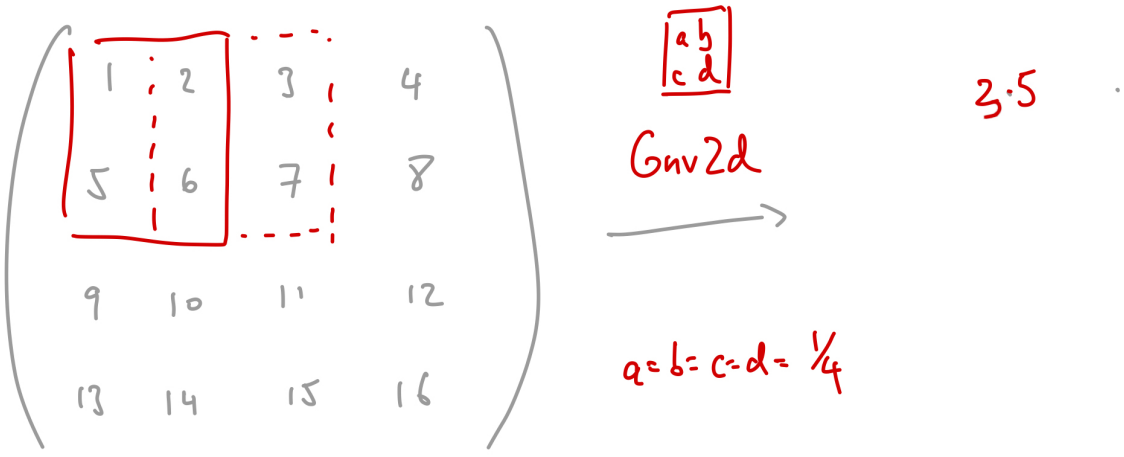


Figure: Move the kernel with some stride (1 here), then continue same process.

Convolutions

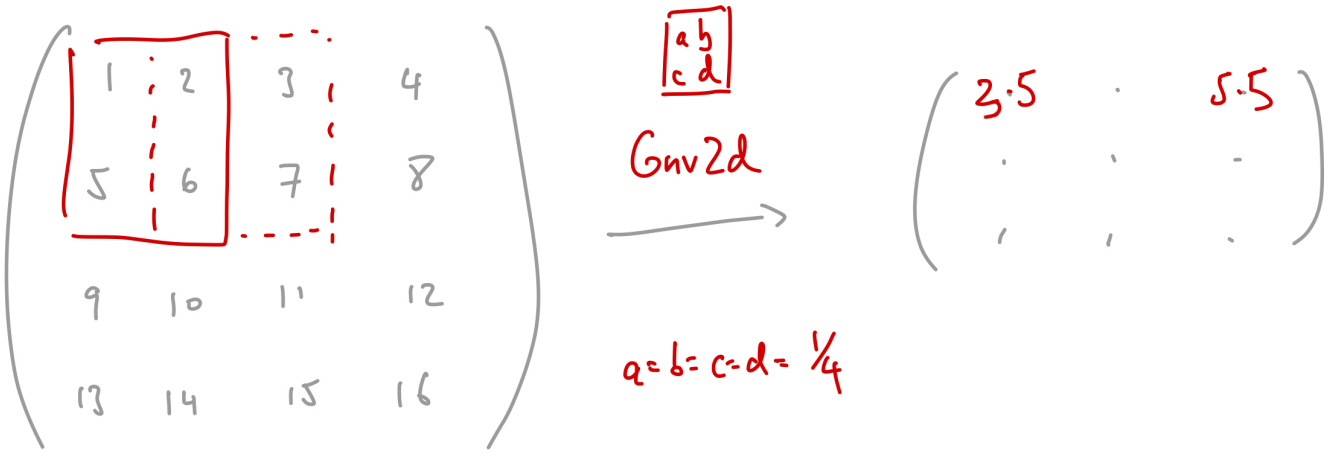


Figure: Repeat until whole array is done. By default convolution leads to an array with **reduced** the size.

Convolutions

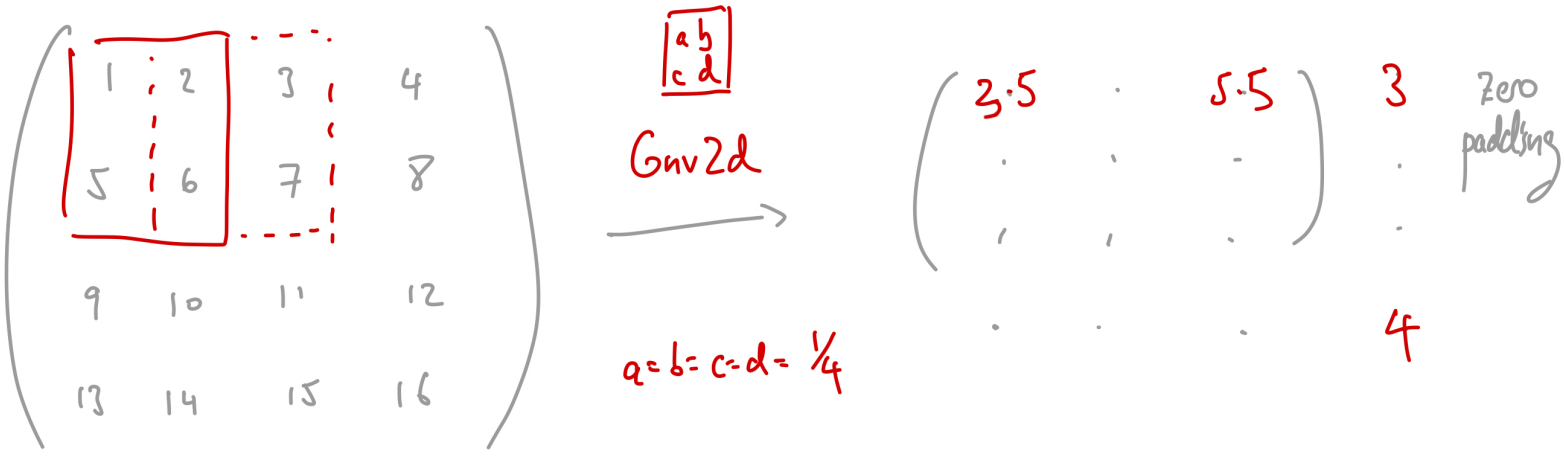


Figure: Can do padding for various reasons. Zero padding here bulks the **input** array size.

Pooling

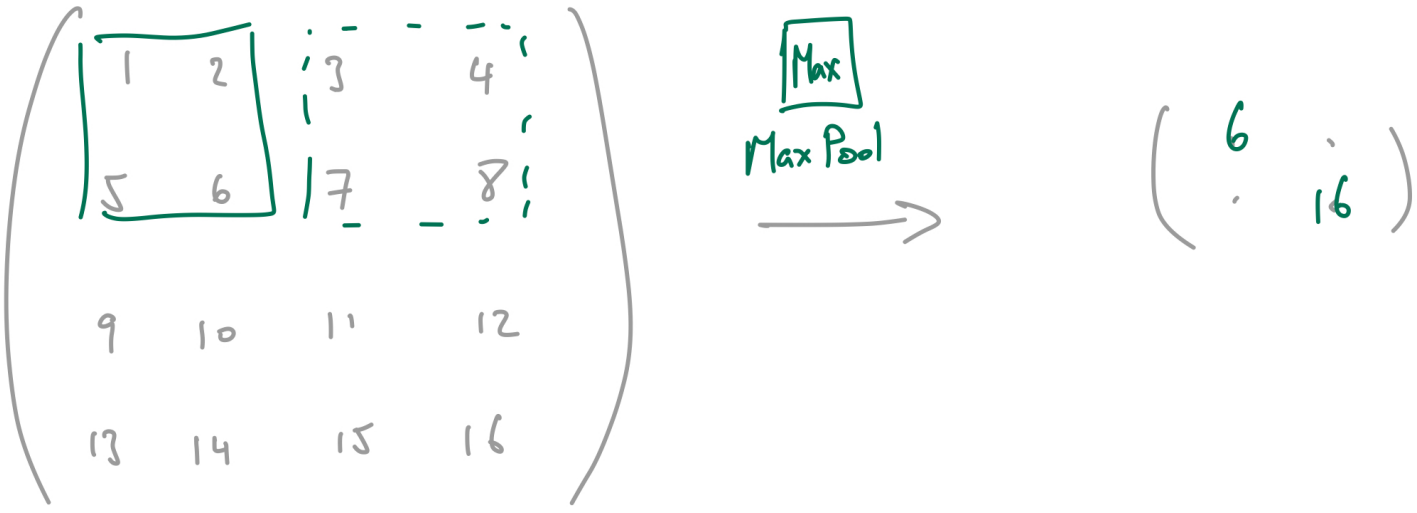


Figure: Pooling is similar, but default has stride equal to kernel size. This also reduces size of array.

- all the above can in principle be done for non-square kernels/poolings and/or unequal strides, paddings (and dilations; look that up if you want)

Convolution example

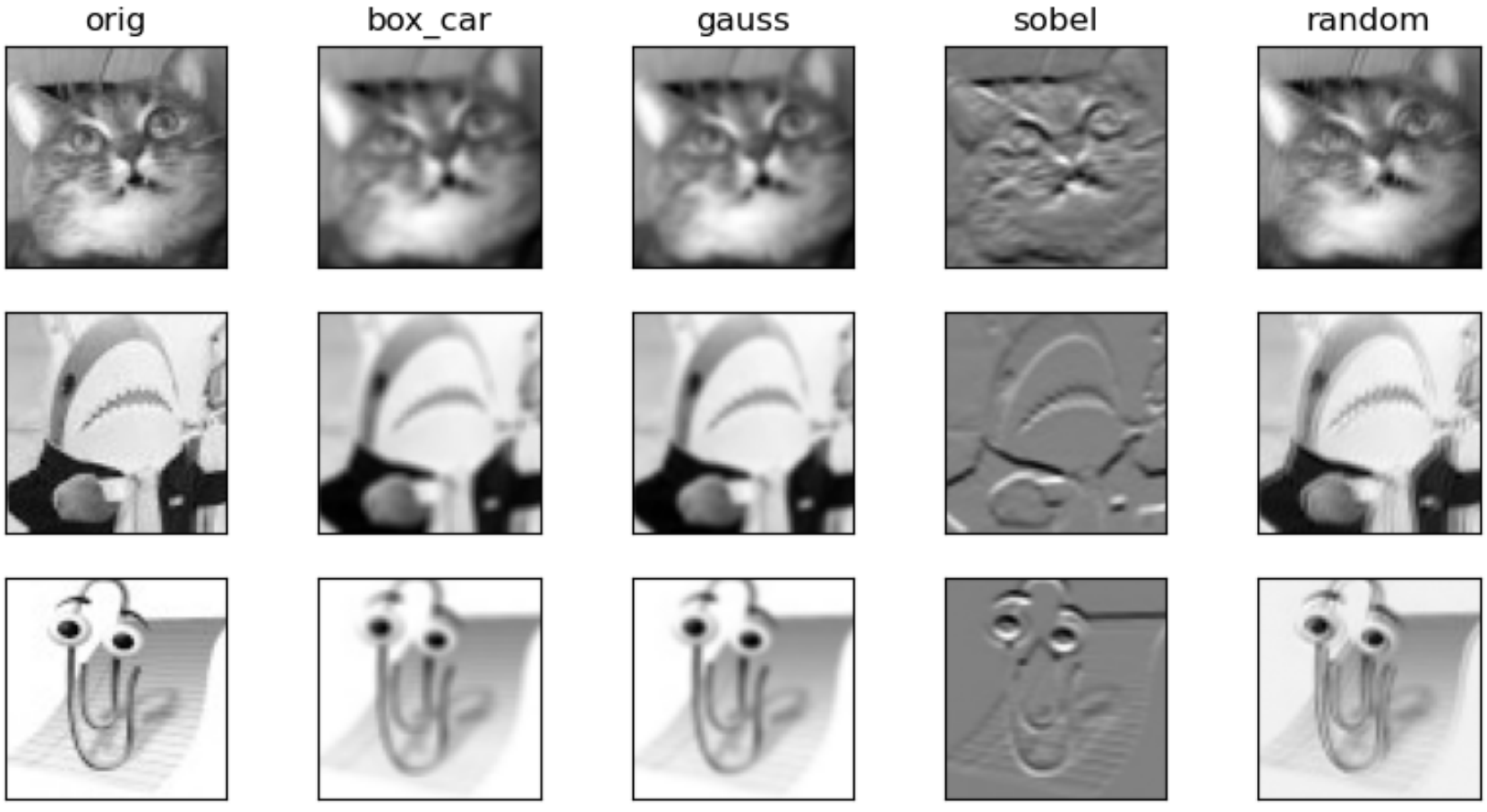


Figure: Convolving the ad-hoc TAs with a size 3 kernel with stride 1 with different choices of kernel weights. Input size is (64, 64) and output size in this case is (62, 62) (or slice 3-1 = 2 pixels off).

Pooling example

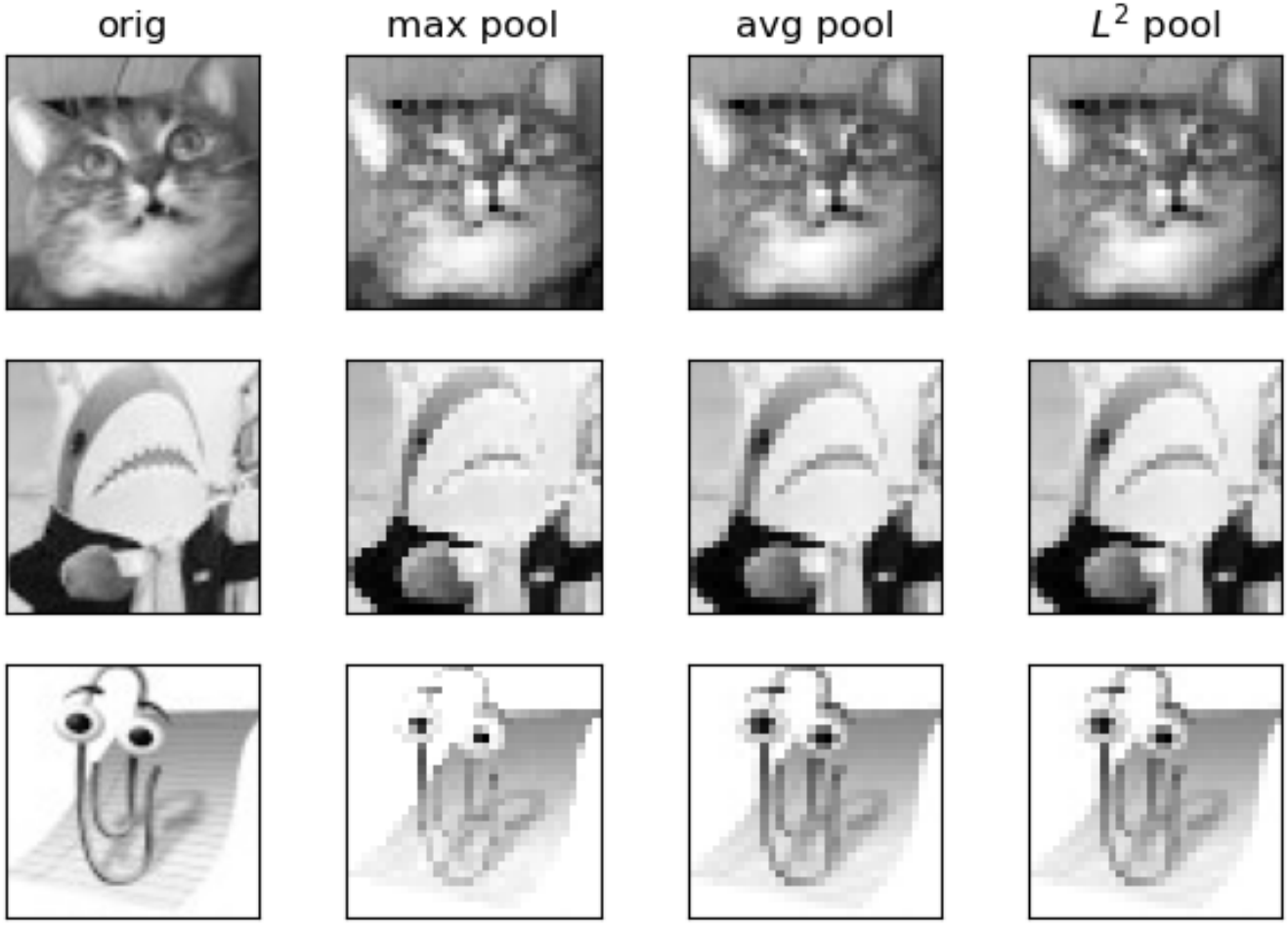


Figure: Pooling the ad-hoc TAs with a size 2 kernel with stride 2 with different operations. Input size is (64, 64) and output size in this case is (31, 31) (or divide by 2 in this case).

CNNs

- CNNs include **kernel weights** as control variables

  → otherwise proceed entirely as before
- controls degrees of freedom a bit better

  → number of kernel weights

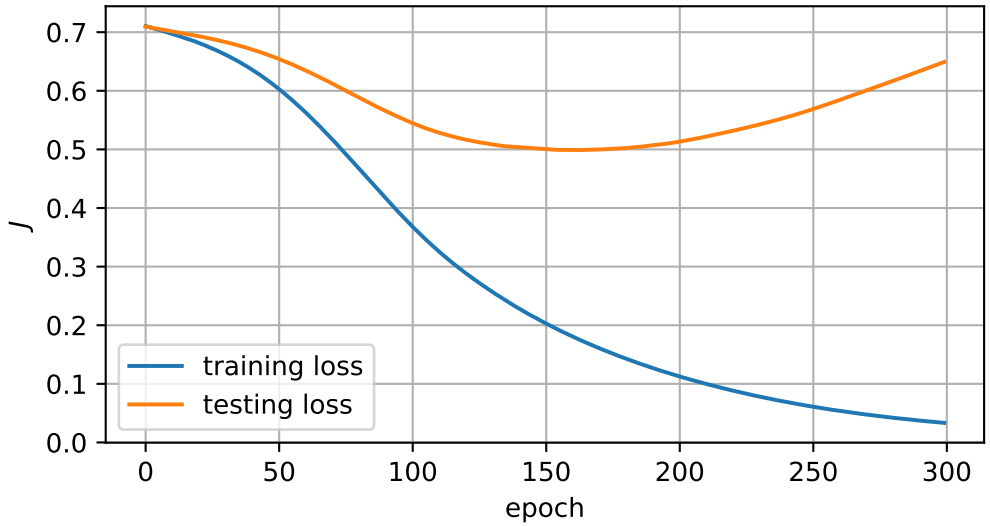
  → reduction in array size

  → linear layer(s) as before

```
        # convolutional layer 1 & max pool layer 1
        self.layer1 = nn.Sequential(
            nn.Conv2d(1, 6, 5),     # now (6, 60, 60)
            nn.ReLU(),
            nn.MaxPool2d((2, 2)),  # now (6, 30, 30)
        )

        # convolutional layer 2 & max pool layer 2
        self.layer2 = nn.Sequential(
            nn.Conv2d(6, 10, 5),    # now (10, 26, 26)
            nn.ReLU(),
            nn.MaxPool2d((2, 2)),  # now (10, 13, 13)
        )

        self.layer3 = nn.Sequential(
            nn.Linear(10 * 13 * 13, 60),
            nn.ReLU(),
            nn.Linear(60, 30),
            nn.ReLU(),
            nn.Linear(30, 2)       # because two possible
        )

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = out.view(-1, 10 * 13 * 13)
        out = self.layer3(out)
        return out
```

Need some care in specifying sizes in the network structure!

CNNs: classification



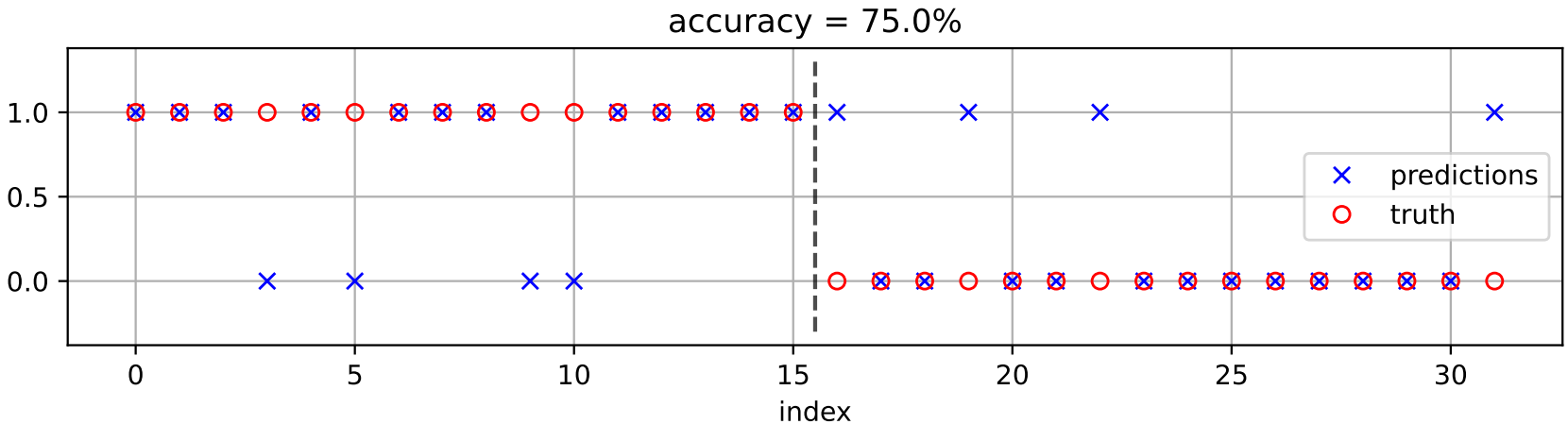


Figure: CNN for classification. Probably hasn't converged, but skill seems ok.

CNNs: regression

- need a change in the loss

  $\rightarrow$ use `MSELoss`
- need to change the CNN structure

  $\rightarrow$ linear layer to expand back to image array

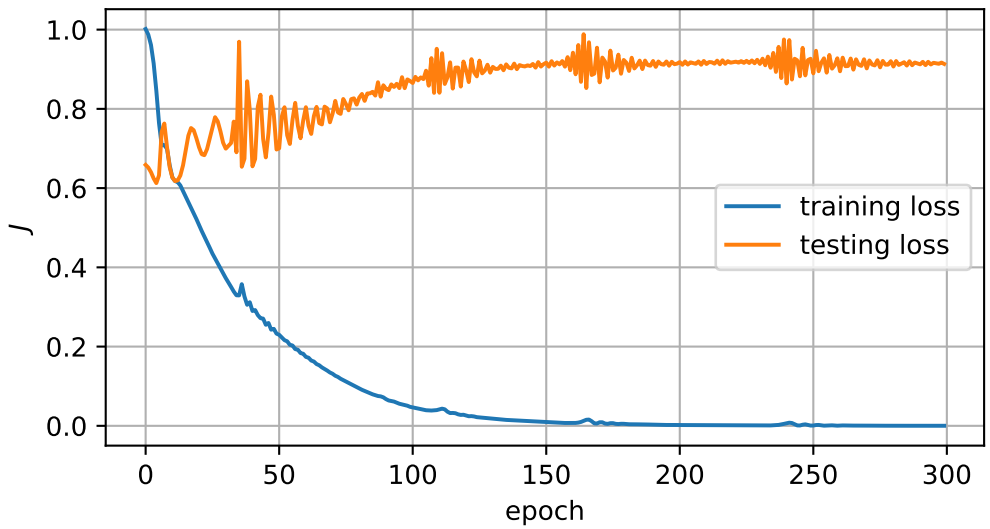


Figure: CNN for regression (predict bottom from top half).

CNNs: regression

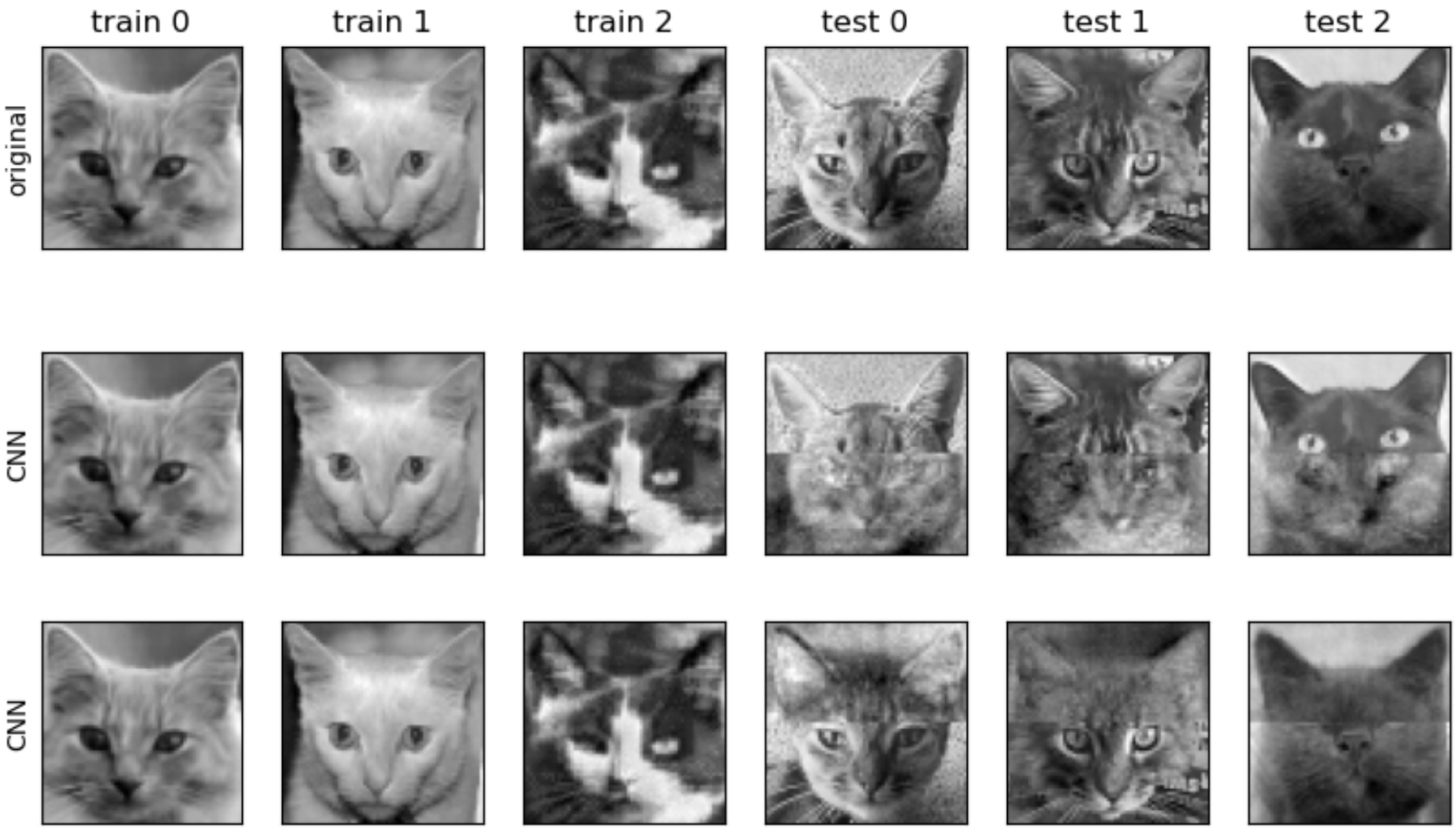


Figure: CNN for regression (predict bottom from top half and vice-versa).

Demonstration

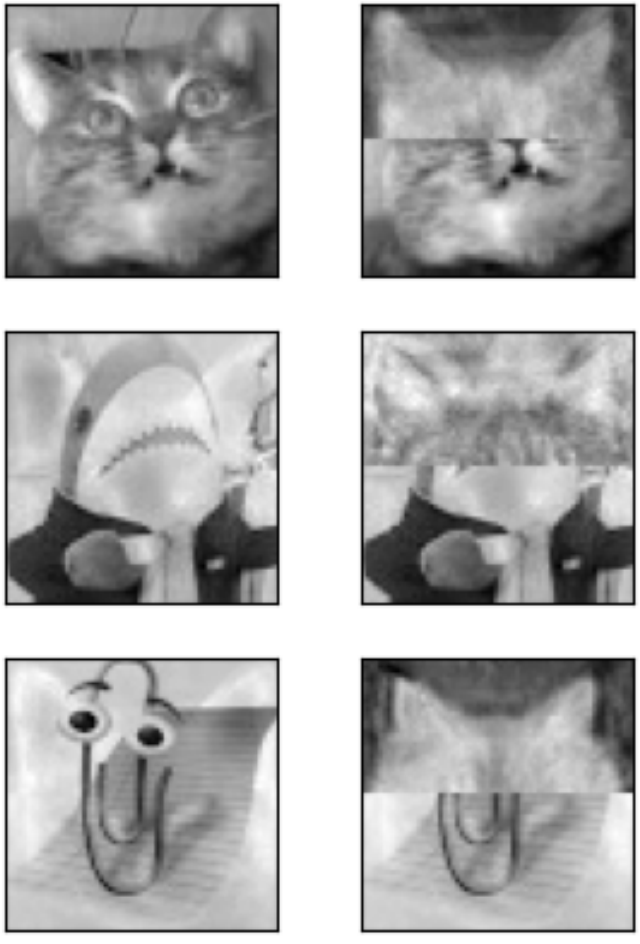


Figure: Grant us eyes (or not).

- introduced the use of PyTorch

  $\rightarrow$ did not use a interface (e.g. Keras), but you can try
- built a CNN

  $\rightarrow$ basics with layer structure etc.

  $\rightarrow$ need hyper-parameter tuning and cross-validation etc.

Moving to a Jupyter notebook $\rightarrow$